Day 09

Error Handling

Phase 3 — Intermediate · 2 Hours · Session Guide

Topics	Prerequisites	Next Session
try/except/else/finally raise & custom exceptions logging module	Day 08 — File I/O Day 05 — Functions Day 06 — Data Structures	Day 10 — OOP I Classes & Objects

By the end of this session students will confidently catch and raise exceptions, design meaningful custom exception classes, and use the **logging** module instead of scattered `print()` statements — skills that are essential for every real-world Python program.

■ LEARNING OBJECTIVES

- ✔ Understand Python's exception hierarchy and why errors are objects
- ✔ Use try / except / else / finally blocks correctly
- ✔ Catch specific exceptions and avoid bare except clauses
- ✔ Raise exceptions manually with raise and informative messages
- ✔ Create custom exception classes by subclassing Exception
- ✔ Replace debug print() statements with the logging module

■ SESSION TIMELINE (120 minutes)

Time	Segment	Activity	Mins
0:00	Warm-up	Recap Day 8, motivating demo — crash without try/except	10
0:10	Concept 1	Exception hierarchy, try/except basics, multiple except clauses	20
0:30	Concept 2	else / finally blocks, re-raising, exception chaining	15
0:45	Live Code	File-reading script with full error handling	15
1:00	Concept 3	raise, assert, custom exception classes	20
1:20	Concept 4	logging module — levels, handlers, formatters	15
1:35	Mini-Project	Bank account validator with custom exceptions + logging	15
1:50	Wrap-up	Recap, Q&A;, Day 10 preview	10

STEP 1 — WARM-UP & MOTIVATING DEMO

0:00 – 0:10 · 10 minutes

What to Say:

> "Yesterday we learned to read and write files. What happens when the file doesn't exist? Python crashes with a traceback. Today we'll learn how to handle that gracefully — catching errors, giving useful messages, and keeping the program running."

Run this deliberately broken code first:

```
# ■■ Motivating Demo — crashes without error handling ■■
f = open('data.txt')   # FileNotFoundError if missing!
content = f.read()
number = int('hello')  # ValueError
result = 10 / 0        # ZeroDivisionError
```

■ Instructor Tip

Let the traceback display in full — point out: exception type, message, and line number. Emphasise that Python errors are objects, not just text. This sets up the hierarchy discussion.

STEP 2 — EXCEPTION HIERARCHY & try/except BASICS

0:10 – 0:30 · 20 minutes

What to Say:

> "Every Python exception is a class. They all inherit from BaseException, but we mostly deal with Exception and its subclasses. Let's look at the hierarchy, then wrap code in a try block."

Exception hierarchy (simplified):

```
BaseException
  ■■■ SystemExit
  ■■■ KeyboardInterrupt
  ■■■ Exception          ← catch this, not BaseException
        ■■■ ValueError
        ■■■ TypeError
        ■■■ FileNotFoundError  (subclass of OSError)
        ■■■ ZeroDivisionError
        ■■■ KeyError
        ■■■ IndexError
        ■■■ AttributeError
```

Basic try/except syntax:

```
# ■■ Basic try/except ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
try:
    number = int(input('Enter a number: '))
    result = 100 / number
    print(f'Result: {result}')
except ValueError:
    print('That was not a valid number!')
except ZeroDivisionError:
    print('Cannot divide by zero!')
except Exception as e:
    print(f'Unexpected error: {e}')
```

■■ Common Mistake — Bare except

Never write 'except:' without a type. It silently catches SystemExit and KeyboardInterrupt, making Ctrl+C stop working. Always specify at least 'except Exception'.

Catching multiple exceptions on one line:

```
# Multiple exceptions in a tuple
try:
    value = int(user_input)
except (ValueError, TypeError) as e:
    print(f'Input error: {e}')
```

Common exceptions reference:

Exception	When It Occurs	Example Trigger
ValueError	Right type, wrong value	`int('hello')`
TypeError	Wrong type for operation	`'a' + 1`
KeyError	Dict key missing	`d['missing']`
IndexError	List index out of range	`lst[99]`
FileNotFoundError	File doesn't exist	`open('x.txt')`
ZeroDivisionError	Division by zero	`10 / 0`
AttributeError	Object lacks attribute	`None.split()`
ImportError	Module not found	`import xyz`

STEP 3 — else / finally BLOCKS

0:30 – 0:45 · 15 minutes

What to Say:

> "The try block has two optional extras. The else block runs only if NO exception was raised — perfect for code that should only run on success. The finally block ALWAYS runs — ideal for cleanup like closing files or database connections."

```
# ■■ Full try / except / else / finally structure ■■■■■■
try:
    file = open('scores.txt', 'r')
    content = file.read()
except FileNotFoundError:
    print('Error: scores.txt not found')
    content = None
else:
    # Runs only when no exception occurred
    print(f'File loaded successfully ({len(content)} chars)')
finally:
    # Always runs — perfect for cleanup
    file.close()  # safe even if open() failed?  No! see below
```

■ Best Practice

Use 'with open(...)' from Day 8 to handle file closing automatically. Reserve 'finally' for closing database connections, network sockets, or releasing locks.

Re-raising and exception chaining:

```
# ■■ Re-raise to propagate after logging ■■■■■■■■■■■■■■■■
try:
    data = load_config('config.json')
except FileNotFoundError as e:
    print('Config missing — using defaults')
    raise  # re-raises the same exception

# ■■ Exception chaining with 'raise ... from ...' ■■■■■■■■
try:
    value = int(raw_data)
except ValueError as e:
    raise TypeError('Expected numeric data') from e
```

STEP 4 — LIVE CODE: File Reader with Error Handling

0:45 – 1:00 · 15 minutes

What to Say:

"Let's combine Day 8 file skills with today's error handling. We'll write a function that reads a CSV of student marks safely — handling missing files, bad data, and empty files."

```
# ■■ safe_read_marks.py ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import csv

def read_marks(filename):
    """Read student marks from a CSV file safely."""
    marks = []
    try:
        with open(filename, newline='', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    marks.append({
                        'name': row['name'],
                        'score': int(row['score'])
                    })
                except (ValueError, KeyError) as e:
                    print(f'Skipping bad row {row}: {e}')
    except FileNotFoundError:
        print(f'File not found: {filename}')
        return []
    if not marks:
        raise ValueError(f'No valid data in {filename}')
    return marks

# Test it
students = read_marks('marks.csv')
print(students)
```

■ Done When

Students can call read_marks() with an existing file, a missing file, and a file with one bad row — seeing different behavior in each case with no unhandled exceptions.

STEP 5 — raise, assert & CUSTOM EXCEPTION CLASSES

1:00 – 1:20 · 20 minutes

What to Say:

> "So far we've caught errors that Python raises. Now let's raise our own — when our code detects invalid input that Python won't catch automatically. And we'll create custom exception classes so callers can catch exactly what went wrong."

Using raise manually:

```
# ■■ raise when business logic is violated ■■■■■■■■■■■■■■■
def set_age(age):
    if not isinstance(age, int):
        raise TypeError(f'Age must be int, got {type(age).__name__}')
    if age < 0 or age > 150:
        raise ValueError(f'Age {age} is outside valid range 0-150')
    return age

set_age(25)    # OK
set_age(-5)    # ValueError: Age -5 is outside valid range 0-150
set_age('old') # TypeError: Age must be int, got str
```

assert for debugging invariants:

```
# assert — for internal invariants (not user input validation)
def calculate_average(scores):
    assert len(scores) > 0, 'scores list cannot be empty'
    return sum(scores) / len(scores)
# Note: assertions can be disabled with python -O flag
```

Custom exception classes:

```python
# ■■ Custom exception hierarchy ■■■■■■■■■■■■■■■■■■■■■■■■■
class BankError(Exception):
    """Base class for all bank-related errors."""
    pass

class InsufficientFundsError(BankError):
    """Raised when withdrawal exceeds balance."""
    def __init__(self, amount, balance):
        self.amount = amount
        self.balance = balance
        super().__init__(
            f'Cannot withdraw Rs {amount}. Balance: Rs {balance}'
        )

class AccountFrozenError(BankError):
    """Raised when operating on a frozen account."""
    pass

# Using custom exceptions
def withdraw(account, amount):
    if account.frozen:
        raise AccountFrozenError('Account is frozen')
    if amount > account.balance:
        raise InsufficientFundsError(amount, account.balance)
    account.balance -= amount

try:
    withdraw(acc, 5000)
except InsufficientFundsError as e:
    print(f'Transaction failed: {e}')
    print(f'You tried to withdraw: Rs {e.amount}')
except BankError as e:
    print(f'Bank error: {e}')
```

■ Design Tip

Create one base exception class per module or domain (e.g. BankError). Then subclass it for specific errors. This lets callers catch all domain errors with one except, or be precise about which error they handle.

STEP 6 — LOGGING MODULE

1:20 – 1:35 · 15 minutes

What to Say:

> "Every real application uses logging instead of print() for diagnostic output. Logging gives us: severity levels, timestamps, file output, and the ability to turn off debug messages in production with one line of config."

```
# ■■ Logging basics ■■■
```

```python
# ■■ Log to file AND console simultaneously ■■■■■■■■■■■■■■■
import logging

logger = logging.getLogger('bank_app')
logger.setLevel(logging.DEBUG)

# File handler — all logs
fh = logging.FileHandler('bank.log')
fh.setLevel(logging.DEBUG)

# Console handler — warnings and above only
ch = logging.StreamHandler()
ch.setLevel(logging.WARNING)

fmt = logging.Formatter('%(asctime)s %(name)s %(levelname)s %(message)s')
fh.setFormatter(fmt)
ch.setFormatter(fmt)

logger.addHandler(fh)
logger.addHandler(ch)
```

Level	Value	When to Use
`DEBUG`	10	Detailed diagnostic info during development
`INFO`	20	Confirmation that things are working as expected
`WARNING`	30	Something unexpected but not an error
`ERROR`	40	A serious problem; some function failed
`CRITICAL`	50	A very serious error; program may not continue

STEP 7 — MINI-PROJECT: Bank Account Validator

1:35 – 1:50 · 15 minutes

What to Say:

> "Let's build a small bank account system that uses custom exceptions and the logging module together. Students implement the BankAccount class and a transaction processor — we'll wire in everything from today's session."

```
# ■■ bank_account.py — exceptions & logging setup ■■■■■■■■
import logging
logging.basicConfig(level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s')
logger = logging.getLogger('bank')

class BankError(Exception): pass

class InsufficientFundsError(BankError):
    def __init__(self, amount, balance):
        self.amount = amount; self.balance = balance
        super().__init__(f'Need Rs {amount}, have Rs {balance}')

class NegativeAmountError(BankError): pass
```

```
# ■■ BankAccount class ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance
        logger.info('Account opened for %s', owner)

    def deposit(self, amount):
        if amount <= 0:
            raise NegativeAmountError('Deposit must be > 0')
        self._balance += amount
        logger.info('Deposited Rs %s. Balance: Rs %s', amount, self._balance)
        return self._balance

    def withdraw(self, amount):
        if amount <= 0:
            raise NegativeAmountError('Withdrawal must be > 0')
        if amount > self._balance:
            raise InsufficientFundsError(amount, self._balance)
        self._balance -= amount
        logger.info('Withdrew Rs %s. Balance: Rs %s', amount, self._balance)
        return self._balance

# ■■ Test ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
acc = BankAccount('Priya', 1000)
acc.deposit(500)
try:
    acc.withdraw(2000)
except InsufficientFundsError as e:
    logger.error('Transaction failed: %s', e)
```

■ Done When

Students have a working BankAccount class that raises custom exceptions for invalid operations, logs all transactions at INFO level, and logs errors at ERROR level — with no bare print() calls.

STEP 8 — WRAP-UP & NEXT DAY PREVIEW

1:50 – 2:00 · 10 minutes

Student Checklist — Can You:

- ■ Write a try/except block catching specific exception types?
- ■ Use else (success path) and finally (cleanup) correctly?
- ■ Raise exceptions manually with raise and a helpful message?
- ■ Create a custom exception class by subclassing Exception?
- ■ Set up logging with basicConfig and use the five log levels?
- ■ Write logging calls that replace debug print() statements?

Key Concepts Recap:

Concept	Key Syntax	Purpose
Basic handling	`try / except ExcType as e`	Catch and respond to errors
Success path	`else:`	Runs only if no exception raised
Cleanup	`finally:`	Always runs (close resources)
Raise manually	`raise ValueError('msg')`	Signal invalid state
Custom exception	`class MyErr(Exception):`	Domain-specific error types
Logging	`logger.info/warning/error()`	Structured diagnostic output

■ NEXT SESSION PREVIEW — Day 10: OOP I — Classes & Objects

Day 10 introduces Object-Oriented Programming. Students will define their first class, use the `__init__` constructor, understand `self`, and distinguish instance vs class variables. The BankAccount class built today is a perfect preview — tomorrow we restructure it as a proper OOP design.

■ QUICK REFERENCE CARD — Error Handling & Logging

```python
# ■■ Full try/except template ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
try:
    risky_operation()
except SpecificError as e:
    handle_it(e)
except (TypeA, TypeB) as e:
    handle_multiple(e)
else:
    run_on_success()
finally:
    always_cleanup()

# ■■ Custom exception ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class AppError(Exception): pass
class SpecificError(AppError):
    def __init__(self, val):
        super().__init__(f'Bad value: {val}')

# ■■ Logging setup ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import logging
logging.basicConfig(level=logging.DEBUG,
    format='%(asctime)s %(levelname)s %(message)s')
logger = logging.getLogger(__name__)
logger.debug/info/warning/error/critical('msg')
```